

第 19 章

特殊工具与技术

导读

本章介绍了一些特殊的 C++ 特性，如控制内存分配、运行时类型识别、枚举类型等。

本章的练习通过贯穿本书的一些例子帮助读者练习这些语言特性的使用。

练习 19.1: 使用 `malloc` 编写你自己的 `operator new(size_t)` 函数，使用 `free` 编写 `operator delete(void *)` 函数。

【出题思路】

用户自定义 `operator new` 函数和 `operator delete` 函数的练习，控制内存分配的过程，不局限于标准库中定义的版本。

【解答】

```
#include <cstdlib>
#include <new>
Using namespace std;

void *operator new(size_t size)
{
    if (void *mem = malloc(size))
    {
        return mem;
    }
    else
    {
        throw bad_alloc();
    }
}
```

```
// 编译时必须指定 C++11, 否则 noexcept 会导致编译错误
void operator delete(void *mem) noexcept
{
    free(mem);
}
```

练习 19.2: 默认情况下, allocator 类使用 operator new 获取存储空间, 然后使用 operator delete 释放它。利用上一题中的两个函数重新编译并运行你的 StrVec 程序 (参见 13.5 节, 第 465 页)。

【出题思路】

使用自定义的全局 operator new 和 operator delete 函数执行分配内存和释放内存的操作, 测试其分配内存的方式是否与标准方式类似。

【解答】

在分配新空间时

```
T *newelements = alloc.allocate(newcapacity);
```

可以重写为:

```
T *newelements = static_cast<T *>
                  (operator new[](newcapacity * sizeof(T)));
```

类似的, 在重新分配由 Vector 成员 elements 指向的旧空间时

```
alloc.deallocate(elements, end - elements);
```

可以重写为:

```
operator delete[](elements);
```

练习 19.3: 已知存在如下的类继承体系, 其中每个类分别定义了一个公有的默认构造函数和一个虚析构函数:

```
class A { /* . . . */ };
class B : public A { /* . . . */ };
class C : public B { /* . . . */ };
class D : public B, public A { /* . . . */ };
```

下面的哪个 dynamic_cast 将失败?

- (a) A *pa = new C;
B *pb = dynamic_cast< B* >(pa);
- (b) B *pb = new B;
C *pc = dynamic_cast< C* >(pb);
- (c) A *pa = new D;
B *pb = dynamic_cast< B* >(pa);

【出题思路】

根据 dynamic_cast 运算符定义中的实际绑定对象类型和目标对象类型之间的要求关系, 判断 dynamic_cast 能否正常转换目标。

【解答】

使用 dynamic_cast 操作符时, 如果运行时实际绑定到引用或指针的对象不是目标类型的对象 (或其公有派生类的对象), 则 dynamic_cast 失败。

(b)dynamic_cast 转换失败。因为目标类型是 C，但 pb 实际指向的不是 C 类对象，而是一个 B 类（C 的基类）对象。

而(c)会编译失败，因为 A 是 D 的一个二义基类。

注意，要使用 RTTI，一般需要在编译器中设置相应编译选项。例如，在 Microsoft Visual C++ .NET 2003 中，在 project 菜单中选择 properties 菜单项，在 configuration properties⇒C/C++->Language 中打开 RTTI 选项。

练习 19.4：使用上一个练习定义的类改写下面的代码，将表达式*pa 转换成类型 C&：

```
if (C *pc = dynamic_cast< C* >(pa))
    // 使用 C 的成员
} else {
    // 使用 A 的成员
}
```

【出题思路】

因引用类型和指针类型的 dynamic_cast 在表示错误发生的方式上有不同之处，通过改写程序，练习不同的异常发生方式。

【解答】

使用 dynamic_cast 将基类引用转换为派生类引用时，如果转换失败，会抛出一个 std::bad_cast 异常，因此可以这样重写上述代码：

```
#include <iostream>
#include <typeinfo>
using namespace std;

class A {
public:
    A() {}
    virtual ~A() {}
};

class B : public A {
public:
    B() {}
    virtual ~B() {}
};

class C : public B {
public:
    C() {}
    virtual ~C() {}
};

int main()
{
    A *pa = new A;
    try
```

```

    {
        C &c = dynamic_cast<C&> (*pa);
    }
    catch (std::bad_cast &bc)
    {
        cout << "dynamic_cast failed" << endl;
    }

    return 0;
}

```

练习 19.5: 在什么情况下你应该使用 `dynamic_cast` 替代虚函数?

【出题思路】

具体情况下, `dynamic_cast` 的使用选择。

【解答】

如果我们需要在派生类中增加新的成员函数 `f`, 但又无法取得基类的源代码, 因而无法在基类中增加相应的虚函数, 这时, 可以在派生类中增加非虚成员函数。但这样一来, 就无法用基类指针调用函数 `f`。如果在程序中需要通过基类指针 (如使用该继承层次的某个类中所包含的指向基类对象的指针数据成员 `p`) 来调用 `f`, 则必须使用 `dynamic_cast` 将 `p` 转换为只想派生类的指针, 才能调用 `f`。也就是说, 如果无法为基类增加虚函数, 就可以使用 `dynamic_cast` 代替虚函数。

练习 19.6: 编写一条表达式将 `Query_base` 指针动态转换为 `AndQuery` 指针 (参见 15.9.1 节, 第 564 页)。分别使用 `AndQuery` 的对象以及其他类型的对象测试转换是否有效。打印一条表示类型转换是否成功的信息, 确保实际输出的结果与期望的一致。

【出题思路】

使用 `dynamic_cast` 动态转换类型的练习, 并测试结果。

【解答】

假设指针 `qb` 的类型为 `Query_base*`, 则表达式 `dynamic_cast<AndQuery*> (qb)` 可动态地将 `Query_base` 对象的指针强制转换为 `AndQuery` 对象的指针。如果强制转换失败, 则转换结果为 0。以下代码可测试转换是否有效:

```

#include <iostream>
using namespace std;

class Query_base
{
public:
    Query_base() {}
    virtual ~Query_base() {}
    // ...
};

class BinaryQuery : public Query_base

```

```

{
public:
    BinaryQuery() {}
    virtual ~BinaryQuery() {}
    // ...
};

class AndQuery : public BinaryQuery
{
public:
    AndQuery() {}
    virtual ~AndQuery() {}
    // ...
};

int main()
{
    Query_base *qb = new Query_base;
    if (dynamic_cast<AndQuery*>(qb) != NULL)
    {
        cout << "success." << endl;
    }
    else
    {
        cout << "failure." << endl;
    }

    return 0;
}

```

练习 19.7：编写与上一个练习类似的转换，这一次将 `Query_base` 对象转换为 `AndQuery` 的引用。重复上面的测试过程，确保转换能正常工作。

【出题思路】

练习使用 `dynamic_cast` 动态转换类型。

【解答】

假设指针 `qb` 的类型为 `Query_base*`，则表达式 `dynamic_cast<AndQuery&>(*qb)` 可动态地将 `Query_base` 对象强制转换为 `AndQuery` 的引用，以下代码可测试转换是否正常工作：

```

#include <iostream>
#include <typeinfo>
using namespace std;

class Query_base
{
public:
    Query_base() {}
    virtual ~Query_base() {}
    // ...
};

```

```

class BinaryQuery : public Query_base
{
public:
    BinaryQuery() {}
    virtual ~BinaryQuery() {}
    // ...
};

class AndQuery : public BinaryQuery
{
public:
    AndQuery() {}
    virtual ~AndQuery() {}
    // ...
};

int main()
{
    Query_base *qb = new Query_base;
    try
    {
        dynamic_cast<AndQuery*>(*qb);
        cout << "success" << endl;
    }
    catch (bad_cast &bc)
    {
        cout << "failure" << endl;
    }

    return 0;
}

```

练习 19.8: 编写一条 typeid 表达式检查两个 Query_base 对象是否指向同一种类型。再检查该类型是否是 AndQuery。

【出题思路】

使用 typeid 运算符进行表达式检查的练习。

【解答】

假设指针 qb1 和 qb2 的类型为 Query_base*，则判断两个 Query_base 指针是否指向相同类型的 typeid 表达式如下：

```
typeid(*qb1) == typeid(*qb2)
```

判断该类型是否为 AndQuery 的 typeid 表达式如下：

```
typeid(*qb1) == typeid(AndQuery)
```

练习 19.9: 编写与本节最后一个程序类似的代码，令其打印你的编译器为一些常见类型所起的名字。如果你得到的输出结果与本书类似，尝试编写一个函数将这些字符串翻译成人们更容易读懂的形式。

【出题思路】

type_info 类的使用练习。

【解答】

```
#include <iostream>
#include <typeinfo>
using namespace std;

class Base {};
class Derived : public Base {};

int main()
{
    Base b, *pb;
    pb = NULL;
    Derived d;

    cout << typeid(int).name() << endl
         << typeid(unsigned).name() << endl
         << typeid(long).name() << endl
         << typeid(unsigned long).name() << endl
         << typeid(char).name() << endl
         << typeid(unsigned char).name() << endl
         << typeid(float).name() << endl
         << typeid(double).name() << endl
         << typeid(string).name() << endl
         << typeid(Base).name() << endl
         << typeid(b).name() << endl
         << typeid(pb).name() << endl
         << typeid(Derived).name() << endl
         << typeid(d).name() << endl
         << typeid(type_info).name() << endl;

    return 0;
}
```

练习 19.10：已知存在如下的类继承体系，其中每个类定义了一个默认公有的构造函数和一个虚析构函数。下面的语句将打印哪些类型名字？

```
class A { /* . . . */ };
class B : public A { /* . . . */ };
class C : public B { /* . . . */ };
(a) A *pa = new C;
    cout << typeid(pa).name() << endl;
(b) C cobj;
    A& ra = cobj;
    cout << typeid(&ra).name() << endl;
(c) B *px = new B;
    A& ra = *px;
    cout << typeid(ra).name() << endl;
```

【出题思路】

熟悉理解 type_info 类对象信息。

【解答】

typeid 操作符的结果是 type_info 类对象, type_info 类的成员函数 name 返回一个 C 风格字符串, 代表 type_info 对象所表示的类型的名字。返回的 C 风格字符串的格式和值根据编译器而定。在 Microsoft Visual Studio 2008 中, 上述语句显示的类型名分别是:

(a) class A*

因为 pa 是指向 A 类对象的指针, 其类型为 A*。

(b) class A*

因为 ra 是 A 类对象的引用, 表达式 &ra 求得 ra 的地址, 该地址的类型为 A*。

(c) class A

因为 ra 是 A 类对象的引用, 其类型为 A。

练习 19.11: 普通的数据指针与指向数据成员的指针有何区别?

【出题思路】

通过和普通数据指针做对比, 理解成员指针。

【解答】

区别在于: 指定指向数据成员的指针的类型时, 除了给出成员本身的类型之外, 还必须给出所属类的类型。例如: 指向 int 型数据的普通数据指针的类型为 int*, 而指向 C 类的 int 型数据成员的成员指针的类型为 int C::*。

练习 19.12: 定义一个成员指针, 令其可以指向 Screen 类的 cursor 成员。通过该指针获得 Screen::cursor 的值。

【出题思路】

本题练习成员指针的定义。

【解答】

指向 Screen 类 cursor 成员的成员指针 pm 可定义如下:

```
pos Screen::* pm = & Screen::cursor;
```

可使用成员指针解引用操作符 (.*) 从对象或引用获取成员, 使用成员指针箭头操作符 (->*) 通过对象的指针获取成员。

假设有如下对象定义:

```
Screen myScreen;
```

则可以这样通过成员指针 pm 获取 Screen::cursor 的值:

```
myScreen .*pm;
```

假设有如下对象指针定义:

```
Screen *pScreen;
```

则可以这样通过成员指针 pm 获取 Screen::cursor 的值:

```
pScreen->*pm;
```


练习 19.13: 定义一个类型, 使其可以表示指向 `Sales_data` 类的 `bookNo` 成员的指针。

【出题思路】

成员指针的相关使用练习。

【解答】

`Sales_data` 类的 `bookNo` 成员是一个数据成员, 其类型为 `std::string`, 可以表示 `Sales_data` 类的 `bookNo` 成员的指针的类型为:

```
std::string Sales_data::*;
```

练习 19.14: 下面的代码合法吗? 如果合法, 代码的含义是什么? 如果不合法, 解释原因。

```
auto pmf = &Screen::get_cursor;
pmf = &Screen::get;
```

【出题思路】

本题练习成员函数指针的使用。

【解答】

合法。

```
auto pmf = &Screen::get_cursor;
// pmf 是一个指向 Screen 成员函数的指针。
pmf = &Screen::get;
// pmf 此时指向 get, get 的版本是根据 pmf 的类型推断出来的, 即无参, 返回 char 的版本。
```

练习 19.15: 普通函数指针和指向成员函数的指针有何区别?

【出题思路】

通过与普通函数指针进行对比, 熟悉理解成员函数指针。

【解答】

区别在于: 指定指向成员函数的指针类型时, 除了给出成员本身的类型之外, 还必须给出成员函数所属类的类型并指明成员函数是否为 `const`。例如: 指向 `int` 型数据的普通数据指针的类型为 `int*`, 指向“不带参数并返回 `int` 型值的函数”的普通函数指针的类型为 `int(*)()`, 而指向“C 类的不带参数并返回 `int` 型值的 `const` 成员函数”的函数成员指针的类型为 `int (C::*)() const`。

练习 19.16: 声明一个类型别名, 令其作为指向 `Sales_data` 的 `avg_price` 成员的指针的同义词。

【出题思路】

本题是成员指针的类型别名定义的练习, 方便程序中对含有成员指针代码的读写。

【解答】

`Sales_data` 的 `avg_price` 成员是一个函数成员, 其原型为:

```
double avg_price() const;
```

定义如下类型别名 Pt, 作为可指向 Sales_data 的 avg_price 成员的指针的同义词:

```
typedef double (Sales_data::*Pt) () const;
// 然后可以像下面这样定义和使用函数指针
Pt pFunc = &Sales_data::avg_price;
Sales_data sd;
(sd.*pFunc)();
```

练习 19.17: 为 Screen 的所有成员函数类型各定义一个类型别名。

【出题思路】

本题练习成员函数类型别名的定义, 方便代码读写。

【解答】

在给出的 Screen 类版本中, 成员函数有 4 个可区分类型, 2 个 get 函数为 2 个不同类型, home 等 5 个光标移动函数为一个类型, move 函数为一个类型。因此可以定义 4 个类型别名 Pmf1、Pmf2、Pmf3 和 Pmf4:

```
typedef char (Screen::*Pmf1) () const;
typedef char (Screen::*Pmf2) (Screen::index, Screen::index) const;
typedef Screen& (Screen::*Pmf3) ();
typedef Screen& (Screen::*Pmf4) (Screen::Directions);
```

练习 19.18: 编写一个函数, 使用 count_if 统计在给定的 vector 中有多少个空 string。

【出题思路】

利用函数对象统计满足条件的对象的个数。

【解答】

首先用 mem_fn 将 string 的 empty 成员函数转换为可调用对象, 将它作为第三个参数传递给 count_if 即可实现统计所有空 string (令 empty 返回真值) 的数量。

```
count_empty_string(vector<string> &vs)
{
    auto f = mem_fn(&string::empty);
    return count_if(vs.begin(), vs.end(), f);
}
```

练习 19.19: 编写一个函数, 令其接受 vector<Sales_data> 并查找平均价格高于某个值的第一个元素。

【出题思路】

练习用 bind 将成员函数转换为可调用对象。

【解答】

`Sales_data` 类的成员函数 `avg_price` 返回平均价格，但它不能将平均价格与某个值进行比较，因此创建函数 `check_value`，比较 `avg_price()` 的返回值和一个给定的值，返回是否平均价格更高。用 `bind` 将 `check_value` 转换为可调用对象（一元谓词），可调用对象的唯一参数（`Sales_data` 对象引用）作为 `check_value` 的第一个参数，用来比较的值作为 `check_value` 的第二个参数。

```
bool check_value(Sales_data &sd, double t)
{
    return sd.avg_price() > t;
}

vector<Sales_data>::iterator find_first_high(vector<Sales_data> &vsd,
double t)
{
    auto f = bind(check_value, _1, t);

    return find_if(vsd.begin(), vsd.end(), f);
}
```

练习 19.20：将你的 `QueryResult` 类嵌套在 `TextQuery` 中，然后重新运行 12.3.2 节（第 435 页）中使用了 `TextQuery` 的程序。

【出题思路】

嵌套类的定义与使用练习。

【解答】

```
class TextQuery {
public:
    class QueryResult;
    // 其他成员的定义
};

class TextQuery::QueryResult {
friend std::ostream&
    print(std::ostream&, const QueryResult&);
public:
    QueryResult(std::string, std::shared_ptr<std::set<line_no>>, std::
        :shared_ptr<std::vector<std::string>>);
};

TextQuery::QueryResult::QueryResult(strings, shared_ptr<set<line_no>>
p, shared_ptr<vector<string>> f):
    sought(s), lines(p), file(f) { }
```

练习 19.21：编写你自己的 `Token` 类。

【出题思路】

通过编写 `Token` 类，练习使用类对 `union` 成员进行管理控制。

【解答】

```
class Token {
```

```

public:
    // 因为 union 含有一个 string 成员，所以 Token 必须定义拷贝控制成员
    Token(): tok(INT), ival{0} { }
    Token(const Token &t): tok(t.tok) { copyUnion(t); }
    Token &operator=(const Token&);
    Token &operator=(char);
    Token &operator=(int);
    Token &operator=(double);
private:
    enum {INT, CHAR, DBL } tok; // 判别式
    union { // 匿名联合
        char cval;
        int ival;
    };
    double dval;
    // 每个 Token 对象含有一个该未命名联合类型的未命名成员
    // 检查判别式，然后酌情拷贝 union 成员
    void copyUnion(const Token&);
};

```

练习 19.22: 为你的 Token 类添加一个 Sales_data 类型的成员。

【出题思路】

练习在类中添加成员，其中包括对成员的管理判别式、赋新值、销毁等操作。

【解答】

```

class Token {
public:
    // 因为 union 含有一个 string 成员，所以 Token 必须定义拷贝控制成员
    // 定义移动构造函数和移动赋值运算符的任务留待本节练习完成
    Token(): tok(INT), ival{0} { }
    Token(const Token &t): tok(t.tok) { copyUnion(t); }
    Token &operator=(const Token&);
    // 如果 union 含有一个 Sales_data 成员，则我们必须销毁它
    ~Token() {if (tok == SDATA) sval.~string(); }
    Token &operator=(char);
    Token &operator=(int);
    Token &operator=(double);
    Token &operator=(Sales_data);
private:
    enum {INT, CHAR, DBL, SDATA } tok; // 判别式
    union { // 匿名联合
        char cval;
        int ival;
        double dval;
        Sales_data sval;
    }; // 每个 Token 对象含有一个该未命名联合类型的未命名成员
    // 检查判别式，然后酌情拷贝 union 成员
    void copyUnion(const Token&);
};

Token &Token::operator=(int i)
{

```

```

// 如果当前存储的是 Sales_data, 释放它
    if (tok == SDATA) sval.~Sales_data();
    ival = i; // 为成员赋值
    tok = INT; // 更新判别式
    return *this;
}

Token &Token::operator=(char c)
{
// 如果当前存储的是 Sales_data, 释放它
    if (tok == SDATA) sval.~Sales_data();
    cval = c; // 为成员赋值
    tok = CHAR; // 更新判别式
    return *this;
}

Token &Token::operator=(double d)
{
// 如果当前存储的是 Sales_data, 释放它
    if (tok == SDATA) sval.~Sales_data();
    dval = d; // 为成员赋值
    tok = DBL; // 更新判别式
    return *this;
}

Token &Token::operator=(const Sales_data &sd)
{
// 如果当前存储的是 Sales_data, 可以直接赋值
    if (tok == SDATA)
        sval = sd;
    else
        new(&sval) Sales_data(sd); // 否则需要先构造一个 Sales_data
    tok = SDATA; // 更新判别式
    return *this;
}

void Token::copyUnion(const Token &t)
{
    switch (t.tok) {
        case Token::INT: ival = t.ival; break;
        case Token::CHAR: cval = t.cval; break;
        case Token::DBL: dval = t.dval; break;
        case Token::SDATA: new(&sval) Sales_data(t.sval); break;
    }
}

Token &Token::operator=(const Token &t)
{
// 如果此对象的值是 string 而 t 的值不是, 则我们必须释放原来的 string
    if (tok == SDATA && t.tok != SDATA) sval.~Sales_data();
    if (tok == SDATA && t.tok == SDATA)
        sval = t.sval; // 无须构造一个新 string

```

```

else
    copyUnion(t);                // 如果 t.tok 是 STR, 则需要构造一个
string
    tok = t.tok;
return *this;
}

```

练习 19.23: 为你的 Token 类添加移动构造函数和移动赋值运算符。

【出题思路】

定义移动构造函数和移动赋值运算符。

【解答】

```

class Token
{
public:
    // 因为 union 含有一个 Sales_data 成员, 所以 Token 必须定义拷贝控制成员
    Token() : tok(INT), ival(0) {}
    Token(const Token &t) : tok(t.tok) { copyUnion(t); }

    // 移动构造函数和移动赋值运算符
    Token(Token &&other);
    Token &operator=(Token &&other);

    Token &operator=(const Token&);
    // 如果 union 含有一个 Sales_data 成员, 则我们必须销毁它
    ~Token() { if (tok == SDATA) sval.~Sales_data (); }
    Token &operator=(char);
    Token &operator=(int);
    Token &operator=(double);
    Token &operator=(Sales_data);

private:
    enum {INT, CHAR, DBL, SDATA} tok;    // 判别式
    union                                // 匿名联合
    {
        char cval;
        int ival;
        double dval;
        Sales_data sval;
    }; // 每个 Token 对象含有一个该匿名联合类型的未命名成员
    // 检查判别式, 然后酌情拷贝 union 成员
    void copyUnion(const Token&);
};

```

Token.cpp 的内容如下:

```

#include "token.h"
Token(Token &&other) : tok(INT), ival(0)
{
    tok = other.tok;
    ival = other.ival;
    other.tok = INT;
}

```

```

        other.ival = 0;
    }

Token &operator=(Token &&other)
{
    if (this != &other)
    {
        delete tok;
        tok = other.tok;
        ival = other.ival;
        other.tok = INT;
        other.ival = 0;
    }

    return *this;
}

Token &Token::operator=(int i)
{
    if (tok == SDATA) // 如果当前存储的是 Sales_data, 释放它
    {
        sval.~Sales_data();
    }

    ival = i;           // 为成员赋值
    tok = INT;          // 更新判别式
    return *this;
}

Token &Token::operator=(char c)
{
    if (tok == SDATA) // 如果当前存储的是 Sales_data, 释放它
    {
        sval.~Sales_data();
    }

    cval = c;           // 为成员赋值
    tok = CHAR;         // 更新判别式
    return *this;
}

Token &Token::operator=(double d)
{
    if (tok == SDATA) // 如果当前存储的是 Sales_data, 释放它
    {
        sval.~Sales_data();
    }

    dval = d;           // 为成员赋值
    tok = DBL;          // 更新判别式
    return *this;
}

```

```

Token &Token::operator=(const Sales_data &sd)
{
    if (tok == SDATA)    // 如果当前存储的是 Sales_data, 可以直接赋值
    {
        sval = sd;
    }
    else                  // 否则需要先构造一个 Sales_data
    {
        new (&sval) Sales_data(sd);
    }

    tok = SDATA;         // 更新判别式
    return *this;
}

void Token::copyUnion(const Token &t)
{
    switch (t.tok)
    {
        case Token::INT:
            ival = t.ival;
            break;

        case Token::CHAR:
            cval = t.cval;
            break;

        case Token::DBL:
            dval = t.dval;
            break;

        case Token::SDATA:
            new(&sval) Sales_data(t.sval);
            break;
    }
}

Token &Token::operator=(const Token &t)
{
    // 如果此对象的值是 Sales_data 而 t 的值不是, 则必须释放原来的 Sales_data
    if (tok == SDATA && t.tok != SDATA)
    {
        sval.~Sales_data();
    }

    if (tok == SDATA && t.tok == SDATA)
    {
        sval = t.sval; // 无须构造一个新 string
    }
    else
    {
        copyUnion(t); // 如果 t.tok 是 STR, 则需要构造一个 string
    }
}

```



```

        tok = t.tok;
        return *this;
    }

```

练习 19.24: 如果我们将一个 Token 对象赋给它自己将发生什么情况？

【出题思路】

熟悉理解移动赋值运算符的原理。

【解答】

因为在移动赋值运算符上添加了检测机制，当一个对象自己给自己赋值时，会先检测，避免不必要或者是灾难性的后果。

练习 19.25: 编写一系列赋值运算符，令其分别接受 union 中各种类型的值。

【出题思路】

熟悉理解赋值运算符的处理机制的原理。

【解答】

```

#include "token.h"

Token(Token &&other) : tok(INT), ival(0)
{
    tok = other.tok;
    ival = other.ival;
    other.tok = INT;
    other.ival = 0;
}

Token &operator=(Token &&other)
{
    if (this != &other)
    {
        delete tok;
        tok = other.tok;
        ival = other.ival;
        other.tok = INT;
        other.ival = 0;
    }

    return *this;
}

Token &Token::operator=(int i)
{
    if (tok == SDATA)    // 如果当前存储的是 Sales_data，释放它
    {
        sval.~Sales_data();
    }

    ival = i;            // 为成员赋值
}

```

```

    tok = INT;                // 更新判别式
    return *this;
}

Token &Token::operator=(char c)
{
    if (tok == SDATA)        // 如果当前存储的是 Sales_data, 释放它
    {
        sval.~Sales_data();
    }

    cval = c;                // 为成员赋值
    tok = CHAR;              // 更新判别式
    return *this;
}

Token &Token::operator=(double d)
{
    if (tok == SDATA)        // 如果当前存储的是 Sales_data, 释放它
    {
        sval.~Sales_data();
    }

    dval = d;                // 为成员赋值
    tok = DBL;               // 更新判别式
    return *this;
}

Token &Token::operator=(const Sales_data &sd)
{
    if (tok == SDATA)        // 如果当前存储的是 Sales_data, 可以直接赋值
    {
        sval = sd;
    }
    else                      // 否则需要先构造一个 Sales_data
    {
        new (&sval) Sales_data(sd);
    }

    tok = SDATA;             // 更新判别式
    return *this;
}

void Token::copyUnion(const Token &t)
{
    switch (t.tok)
    {
        case Token::INT:
            ival = t.ival;
            break;

        case Token::CHAR:
            cval = t.cval;

```

```

        break;

    case Token::DBL:
        dval = t.dval;
        break;

    case Token::SDATA:
        new(&sval) Sales_data(t.sval);
        break;
    }
}

Token &Token::operator=(const Token &t)
{
    // 如果此对象的值是 Sales_data 而 t 的值不是, 则必须释放原来的 Sales_data
    if (tok == SDATA && t.tok != SDATA)
    {
        sval.~Sales_data();
    }

    if (tok == SDATA && t.tok == SDATA)
    {
        sval = t.sval; // 无须构造一个新 string
    }
    else
    {
        copyUnion(t); // 如果 t.tok 是 STR, 则需要构造一个 string
    }

    tok = t.tok;
    return *this;
}

```

练习 19.26: 说明下列声明语句的含义并判断它们是否合法:

```

extern "C" int compute(int *, int);
extern "C" double compute(double *, double);

```

【出题思路】

理解链接指示的声明, 熟悉函数重载的使用。

【解答】

第一个声明指出: `compute` 是一个用 C 语言编写的函数, 该函数接受一个 `int*` 类型及一个 `int` 类型的形参, 返回 `int` 型值。

第二个声明指出: `compute` 是一个用 C 语言编写的函数, 该函数接受一个 `double*` 类型及一个 `double` 类型的形参, 返回 `double` 型值。

如果这两个声明单独出现, 则是合法的; 如果二者同时出现, 则是不合法的, 因为这两个 `compute` 函数构成了函数重载, 而 C 语言是不支持函数重载的。